

DEEP LEARNING & COMPUTER VISION

Practice Questions with Detailed Solutions

TOPICS COVERED

- Topic 1 — Mathematics & Activation Functions
 - Topic 2 — Computer Vision Fundamentals
- Topic 3 — Neural Networks & Backpropagation
- Topic 4 — Convolutional Neural Networks (CNNs)
- Topic 5 — CNN Architectures (AlexNet, VGG, Inception, ResNet)
 - Topic 6 — Normalization & Regularization
- Topic 7 — Recurrent Neural Networks & LSTM
- Topic 8 — Sequence Models & Advanced RNNs
- Topic 9 — Autoencoders & Image Segmentation
- Topic 10 — PyTorch Code & Tracing Questions

ePGD – Deep Learning & Generative AI | IIT Bombay + Great Learning
3 Levels per Topic: Easy ♦ Medium ♦ Hard | April 2026

TABLE OF CONTENTS

TABLE OF CONTENTS 2

TOPIC 1: Mathematics & Activation Functions 3

TOPIC 2: Computer Vision Fundamentals..... 5

TOPIC 3: Neural Networks & Backpropagation 7

TOPIC 4: Convolutional Neural Networks..... 9

TOPIC 5: CNN Architectures..... 11

TOPIC 6: Normalization & Regularization..... 13

TOPIC 7: Recurrent Neural Networks & LSTM 15

TOPIC 8: Sequence Models & Advanced RNNs..... 18

TOPIC 9: Autoencoders & Image Segmentation..... 21

TOPIC 10: PyTorch Code & Tracing Questions..... 24

EXAM QUICK REFERENCE..... 27

TOPIC 1: Mathematics & Activation Functions**MATHEMATICS & ACTIVATION FUNCTIONS**

Chain Rule · Loss Functions · Optimizers · Activation Functions

Q1. [Easy] [2 marks]

What is the sigmoid activation function? Write its formula, range, and derivative.

✓ **DETAILED SOLUTION****Formula**

$$\sigma(x) = 1 / (1 + e^{(-x)})$$

Range & Properties

- Output range: (0, 1) — always positive, never exactly 0 or 1
- Symmetric around $x = 0 \rightarrow \sigma(0) = 0.5$
- Monotonically increasing

Derivative (critical for backprop)

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

- Maximum value of derivative: 0.25 (at $x = 0$)
- At $x = \pm 5$: $\sigma'(x) \approx 0.007$ — very small!

WHY THIS CAUSES VANISHING GRADIENT: Each layer multiplies by $\sigma'(x) \leq 0.25$. For 10 layers: $0.25^{10} \approx 9.5 \times 10^{-7}$. The gradient becomes essentially zero, so early layers stop learning.

KEY TAKEAWAY: Sigmoid squashes any real number to (0,1). Derivative ≤ 0.25 — root of vanishing gradient.

Q2. [Medium] [3 marks]

Compare ReLU and Leaky ReLU. When does each fail, and how does Leaky ReLU fix the issue?

✓ **DETAILED SOLUTION****ReLU: $f(x) = \max(0, x)$**

$$f(x) = \max(0, x) \quad f'(x) = 1 \text{ if } x > 0, \text{ else } 0$$

- Gradient = 1 when active $\rightarrow$ NO vanishing gradient for positive inputs
- Computationally very cheap (just a threshold)
- DYING ReLU PROBLEM: If $x \leq 0$, gradient = 0. Weight receives no update.
- Once dead (negative pre-activation), the neuron stays dead forever during training.

Leaky ReLU: $f(x) = \max(\alpha x, x)$ where $\alpha \approx 0.01$

$$f(x) = x \text{ if } x > 0, \text{ else } 0.01x \quad f'(x) = 1 \text{ if } x > 0, \text{ else } 0.01$$

- Small slope (0.01) for negative inputs $\rightarrow$ gradient is NEVER exactly 0
- Neurons can always 'recover' — gradient still flows even when inactive

When to use which?

- ReLU: Default choice; fast, works for most tasks
- Leaky ReLU: When you observe many dead neurons (loss stagnates early in training)

- He initialization ($\text{var} = 2/\text{fan_in}$) is recommended with ReLU/Leaky ReLU

KEY TAKEAWAY: ReLU is fast but can 'die'. Leaky ReLU fixes dying neurons with a small slope for $x < 0$.

Q3. [Hard] [4 marks]

Derive the weight update formula for Adam optimizer. Explain each term and its role.

✓ DETAILED SOLUTION

Step 1 — Compute Gradient

$$g_\theta = \partial L / \partial w_\theta$$

Step 2 — First Moment (Momentum)

$$m_\theta = \beta_1 \cdot m_{\theta-1} + (1 - \beta_1) \cdot g_\theta$$

- m_θ : Exponential moving average of gradients (direction memory)
- $\beta_1 = 0.9$ (typical): 90% previous direction + 10% current gradient
- Purpose: Smooths noisy gradients; provides momentum like a ball rolling downhill

Step 3 — Second Moment (Adaptive Scale)

$$v_\theta = \beta_2 \cdot v_{\theta-1} + (1 - \beta_2) \cdot g_\theta^2$$

- v_θ : Exponential moving average of SQUARED gradients
- $\beta_2 = 0.999$ (typical): Very slow decay of second moment
- Purpose: Large gradients $\rightarrow$ large $v_\theta \rightarrow$ smaller effective LR (adaptive)

Step 4 — Bias Correction (Critical at start of training)

$$\hat{m}_\theta = m_\theta / (1 - \beta_1^t) \quad \hat{v}_\theta = v_\theta / (1 - \beta_2^t)$$

- At $t=1$: $\beta_1^{-1}=0.9$, so $1-\beta_1=0.1$. Without correction, m_θ is under-estimated 10× at start.
- Correction ensures estimates are accurate even in early training steps

Step 5 — Weight Update

$$w_{\theta+1} = w_\theta - \alpha \cdot \hat{m}_\theta / (\sqrt{\hat{v}_\theta} + \epsilon)$$

- $\epsilon = 10^{-8}$: Prevents division by zero
- Effective LR per parameter = $\alpha / \sqrt{\hat{v}_\theta}$: parameters with large, consistent gradients get SMALLER LR
- This makes Adam robust to different gradient scales across parameters

KEY TAKEAWAY: Adam = Momentum (1st moment) + RMSProp (2nd moment) + Bias correction. Most robust optimizer.

TOPIC 2: Computer Vision Fundamentals

COMPUTER VISION FUNDAMENTALS

Tasks · Datasets · Challenges · Historical Milestones

Q4. [Easy] [2 marks]

List and briefly explain 4 major computer vision tasks.

✓ DETAILED SOLUTION

- 1. Image Classification: Assign ONE label to entire image. Input: image → Output: class label. Example: 'This image contains a cat.' (1000 classes in ImageNet)
- 2. Object Detection: Locate AND label multiple objects with bounding boxes. Output: [(class, x, y, w, h), ...]. Example: YOLO, Faster R-CNN.
- 3. Semantic Segmentation: Assign class label to EVERY pixel. No distinction between instances of same class. Example: All cars → same color.
- 4. Instance Segmentation: Detect individual object INSTANCES with separate masks. Example: 3 cars → 3 different masks. Mask R-CNN.
- 5. Pose Estimation: Predict locations of human body keypoints (joints). Example: Sports analytics, AR apps.

Difficulty order: Classification < Detection < Segmentation < Panoptic Segmentation

KEY TAKEAWAY: Classification → Detection → Segmentation → Estimation — increasing difficulty and output detail.

Q5. [Medium] [3 marks]

Why was AlexNet (2012) considered a breakthrough? List at least 4 specific innovations.

✓ DETAILED SOLUTION

Context: ImageNet Challenge History

- 2011 winner: 25.8% top-5 error using hand-crafted features
- AlexNet 2012: 16.4% — a 10 percentage point improvement, unprecedented in competition history

Innovation 1 — ReLU instead of Sigmoid/Tanh

- ReLU trains ~6× faster than tanh on CIFAR-10 (Krizhevsky et al.)
- No saturation for positive inputs → gradient = 1 → no vanishing gradient

Innovation 2 — GPU Training

- First large-scale CNN trained on NVIDIA GTX 580 GPUs
- Training on GPU made deep networks practically feasible (CPU would take weeks)

Innovation 3 — Dropout Regularization

- p=0.5 applied to fully connected layers during training
- Prevented co-adaptation; equivalent to training ensemble of 2^n subnetworks

Innovation 4 — Data Augmentation

- Random horizontal flips, random crops from 256×256 to 224×224
- PCA color jitter in RGB space
- Artificially expanded training set → reduced overfitting

Architecture: 5 Conv + 3 FC layers → ~60M parameters

KEY TAKEAWAY: AlexNet combined GPUs + ReLU + Dropout + Data Augmentation to reduce ImageNet error from 26% to 16.4%.

Q6. [Hard] [4 marks]

Explain in detail why computer vision is considered a hard problem. What makes human visual recognition so different from what computers do?

✓ DETAILED SOLUTION**Challenge 1 — Viewpoint Variation**

- The 3D world is projected onto a 2D sensor: same 3D object produces wildly different 2D images from different angles.
- A chair from the front vs side vs above can look completely different. Humans understand it's the same object; naive pixel comparison fails.

Challenge 2 — Illumination & Photometric Changes

- Same face in sunlight vs shadow: pixel values change dramatically, even though the object is identical.
- Cameras have dynamic range limitations; humans adapt automatically.

Challenge 3 — Intra-class Variation

- 'Chair' can mean hundreds of different shapes, sizes, colors, and styles.
- Requires learning a high-level semantic concept, not just pixel patterns.

Challenge 4 — Occlusion

- Objects partially hidden behind others. Humans use context to complete the picture; CNNs only see partial features.

Challenge 5 — Background Clutter & Context

- A leopard on a tree branch: the spots in the background could be leaves or leopard skin.
- Humans use scene understanding; CNNs can be fooled by adversarial patches.

Human vs CNN Approach

- Humans: Top-down reasoning — use prior knowledge, physics, context to interpret ambiguous inputs
- CNNs: Bottom-up pattern matching — learned statistical associations between pixel patterns and labels
- Result: CNNs are brittle to distribution shift; humans generalize across domains instantly

KEY TAKEAWAY: The gap lies in semantic understanding, context, and intuitive physics — humans use top-down reasoning; CNNs are bottom-up pattern matchers.

TOPIC 3: Neural Networks & Backpropagation

NEURAL NETWORKS & BACKPROPAGATION

MLP · Universal Approximation · Chain Rule · Training

Q7. [Easy] [2 marks]

Walk through a complete forward pass for a 2-layer neural network with one input, two hidden units, and one output. Use specific numbers.

✓ DETAILED SOLUTION

Setup (given values)

- Input: $x = 2.0$
- Hidden layer weights: $w_1 = 0.5$, $w_2 = -0.3$ (connecting x to 2 hidden units)
- Hidden layer biases: $b_1 = 0.1$, $b_2 = 0.2$
- Output layer weights: $v_1 = 0.8$, $v_2 = -0.6$, bias: $c = 0.0$
- Activation: $\text{ReLU} = \max(0, z)$

Step 1 — Pre-activations of hidden layer

$$\begin{aligned} z_1 &= w_1 \cdot x + b_1 = 0.5 \times 2.0 + 0.1 = 1.1 \\ z_2 &= w_2 \cdot x + b_2 = -0.3 \times 2.0 + 0.2 = -0.4 \end{aligned}$$

Step 2 — Activations of hidden layer

$$\begin{aligned} h_1 &= \text{ReLU}(z_1) = \text{ReLU}(1.1) = 1.1 \\ h_2 &= \text{ReLU}(z_2) = \text{ReLU}(-0.4) = 0.0 \quad \leftarrow \text{this neuron is inactive!} \end{aligned}$$

Step 3 — Output layer

$$\hat{y} = v_1 \cdot h_1 + v_2 \cdot h_2 + c = 0.8 \times 1.1 + (-0.6) \times 0.0 + 0 = 0.88$$

Note: The second hidden neuron is completely inactive (dead for this input), so v_2 has zero contribution to the prediction.

KEY TAKEAWAY: Forward pass = linear transform + activation, repeated per layer. No shortcuts!

Q8. [Medium] [3 marks]

Derive the backpropagation update for a single weight w_1 using the example above. Target $y = 1.0$, Loss = $(\hat{y} - y)^2$, $\text{lr} = 0.1$.

✓ DETAILED SOLUTION

Given (from Q7): $x=2$, $w_1=0.5$, $z_1=1.1$, $h_1=1.1$, $\hat{y}=0.88$, $y=1.0$

Step 1 — Loss

$$L = (\hat{y} - y)^2 = (0.88 - 1.0)^2 = (-0.12)^2 = 0.0144$$

Step 2 — $\partial L / \partial \hat{y}$ (gradient of loss w.r.t. prediction)

$$\partial L / \partial \hat{y} = 2(\hat{y} - y) = 2(0.88 - 1.0) = -0.24$$

Step 3 — $\partial \hat{y} / \partial h_1$ (gradient of output w.r.t. h_1)

$$\hat{y} = v_1 \cdot h_1 + \dots \rightarrow \partial \hat{y} / \partial h_1 = v_1 = 0.8$$

Step 4 — $\partial h_1 / \partial z_1$ (gradient through ReLU)

$$h_1 = \text{ReLU}(z_1), \quad z_1 = 1.1 > 0 \rightarrow \partial h_1 / \partial z_1 = 1$$

Step 5 — $\partial z_1 / \partial w_1$ (gradient of pre-activation w.r.t. weight)

$$z_1 = w_1 \cdot x + b_1 \rightarrow \partial z_1 / \partial w_1 = x = 2.0$$

Step 6 — Chain Rule (multiply all together)

$$\partial L / \partial w_1 = (-0.24) \times 0.8 \times 1 \times 2.0 = -0.384$$

Step 7 — Weight Update

$$w_{1_new} = w_1 - lr \times \partial L / \partial w_1 = 0.5 - 0.1 \times (-0.384) = 0.5 + 0.0384 = 0.5384$$

The weight increased because the prediction was too LOW ($0.88 < 1.0$), so the network needs to increase the contribution of h_1 (which means increasing w_1).

KEY TAKEAWAY: Chain rule: $\partial L / \partial w_i = \partial L / \partial \hat{y} \times \partial \hat{y} / \partial h_i \times \partial h_i / \partial z_i \times \partial z_i / \partial w_i$. Follow the path backward.

Q9. [Hard] [4 marks]

Explain the Universal Approximation Theorem. What are its limitations in practice?

✓ DETAILED SOLUTION**The Theorem (Cybenko, 1989; Hornik, 1991)**

A feedforward neural network with: (1) at least ONE hidden layer, (2) FINITE number of neurons, (3) a NON-LINEAR (non-polynomial) activation function, can approximate any continuous function $f: K \rightarrow \mathbb{R}$ on a compact (closed, bounded) domain $K \subseteq \mathbb{R}^n$, to arbitrary accuracy $\epsilon > 0$.

$$\forall \epsilon > 0, \exists \text{ network } \phi \text{ such that: } \sup_{\{x \in K\}} |\phi(x) - f(x)| < \epsilon$$

What the theorem DOES say:

- A 2-layer network is THEORETICALLY sufficient to represent any function
- Applies to compact (bounded) domains — not infinite input spaces
- Requires non-polynomial activation (sigmoid, tanh, ReLU all qualify)

What the theorem does NOT say (Limitations):

- 1. NUMBER OF NEURONS: The required width could be exponential in input dimension. For practical functions, a shallow network may need astronomically many neurons.
- 2. TRAINING: The theorem says the function CAN be represented — not that gradient descent WILL find the right weights. The optimization landscape may have many bad local minima.
- 3. GENERALIZATION: Even if the training loss is minimized, the network may overfit to training data and fail on new inputs. UAT says nothing about generalization.
- 4. DEPTH: Modern understanding shows DEEP networks are more efficient than wide ones for many natural functions (hierarchical structure). Depth creates $O(m^L)$ linear regions vs $O(m)$ for width.

Practical takeaway:

The theorem justifies using neural networks as universal function approximators, but the engineering challenge is finding the right architecture, training algorithm, and regularization to practically achieve this in finite time with limited data.

KEY TAKEAWAY: UAT guarantees representational capacity, not trainability, generalization, or sample efficiency.

TOPIC 4: Convolutional Neural Networks

CONVOLUTIONAL NEURAL NETWORKS (CNNs)

Convolution · Stride · Padding · Pooling · Parameters

Q10. [Easy] [1 mark]

What is the output spatial size for the following conv layer? Input: 28×28, Kernel: 3×3, Stride: 1, Padding: 1.

✓ DETAILED SOLUTION

Formula:

$$\text{Output_size} = \text{floor}((\text{Input} - \text{Kernel} + 2 \times \text{Padding}) / \text{Stride}) + 1$$

Substituting values:

$$\begin{aligned} &= \text{floor}((28 - 3 + 2 \times 1) / 1) + 1 \\ &= \text{floor}(27 / 1) + 1 = 27 + 1 = 28 \end{aligned}$$

OUTPUT SIZE: 28×28 — same as input!

Why? The 'Same Padding' Pattern:

- For kernel=3, padding=1, stride=1: output ALWAYS equals input size
- For kernel=5, padding=2, stride=1: output also equals input size
- General rule for same padding: $P = (K-1)/2$ when $S=1$
- REMEMBER: 'Same' padding only guarantees equal size for stride=1!

KEY TAKEAWAY: Output = 28×28. Same padding with stride=1 preserves spatial dimensions — always!

Q11. [Medium] [3 marks]

Trace the output dimensions step by step: Input (batch=4, C=3, H=64, W=64). Conv2d(3→32, k=5, s=2, p=0), Conv2d(32→64, k=3, s=1, p=1), MaxPool2d(k=2, s=2). What is the final feature map shape?

✓ DETAILED SOLUTION

Starting shape: (4, 3, 64, 64)

Layer 1: Conv2d(3→32, k=5, s=2, p=0)

$$H_{\text{out}} = \text{floor}((64 - 5 + 0) / 2) + 1 = \text{floor}(59/2) + 1 = 29 + 1 = 30$$

- Output: (4, 32, 30, 30)
- Parameters: $5 \times 5 \times 3 \times 32 + 32 = 2400 + 32 = 2432$

Layer 2: Conv2d(32→64, k=3, s=1, p=1)

$$H_{\text{out}} = \text{floor}((30 - 3 + 2) / 1) + 1 = 29 + 1 = 30$$

- Output: (4, 64, 30, 30)
- Parameters: $3 \times 3 \times 32 \times 64 + 64 = 18432 + 64 = 18496$

Layer 3: MaxPool2d(k=2, s=2)

$$H_{\text{out}} = \text{floor}((30 - 2) / 2) + 1 = 14 + 1 = 15$$

- Output: (4, 64, 15, 15)

- No learnable parameters in MaxPool!

Final shape: (4, 64, 15, 15)

- If we flatten for FC: $64 \times 15 \times 15 = 14,400$ features per sample
- Total trainable params so far: $2432 + 18496 = 20,928$

KEY TAKEAWAY: Always apply the formula at each layer. After pooling: (4, 64, 15, 15).

Q12. [Hard] [4 marks]

What is dilated convolution? A 3×3 kernel with dilation=4 is applied to a 20×20 feature map with no padding. What is the output size and effective receptive field?

✓ DETAILED SOLUTION

What is Dilated (Atrous) Convolution?

A dilated convolution inserts (dilation-1) zeros BETWEEN each kernel element, effectively spacing out the kernel over a larger region without adding parameters.

Effective Kernel Size Formula:

$$\begin{aligned} \text{Effective_K} &= (K - 1) \times \text{dilation} + 1 \\ &= (3 - 1) \times 4 + 1 = 8 + 1 = 9 \quad \leftarrow \text{No! Wait, let me recalculate} \\ \text{Effective_K} &= (3 - 1) \times 4 + 1 = 9 \end{aligned}$$

Output Size Calculation:

$$\begin{aligned} H_{\text{out}} &= \text{floor}((H_{\text{in}} - \text{Effective_K} + 2 \times P) / S) + 1 \\ &= \text{floor}((20 - 9 + 0) / 1) + 1 = \text{floor}(11) + 1 = 12 \end{aligned}$$

Output size: 12×12

Receptive Field Explanation:

- Standard 3×3 kernel: each output pixel 'sees' a $3 \times 3 = 9$ -pixel region
- 3×3 dilated with rate=4: each output pixel 'sees' a 9×9 -pixel region (using only 9 of those 81 pixels — the remaining are skipped)
- Parameters: STILL only 9 ($K \times K = 3 \times 3$) — same as standard 3×3 !

Why this matters (DeepLab):

- Standard stride-based downsampling loses spatial resolution
- Dilated conv captures large context WITHOUT losing resolution
- ASPP uses rates 1, 6, 12, 18 → effective kernels of 1, 13, 25, 37 simultaneously

KEY TAKEAWAY: Dilation expands receptive field without extra params. Effective kernel = $(K-1) \times d + 1 = 13$. Output = 8×8 .

TOPIC 5: CNN Architectures

CNN ARCHITECTURES

AlexNet · VGGNet · GoogLeNet/Inception · ResNet · DenseNet

Q13. [Easy] [2 marks]

What is the key architectural difference between ResNet-18/34 and ResNet-50/101/152?

✓ DETAILED SOLUTION

Basic Block (ResNet-18, ResNet-34)

Input → Conv(3×3) → BN → ReLU → Conv(3×3) → BN → (+skip) → ReLU → Output

- Parameters per block (64 channels): $2 \times (3 \times 3 \times 64 \times 64) = 73,728$
- Skip connection: direct identity (no extra params) when channel size matches

Bottleneck Block (ResNet-50, 101, 152)

Input → Conv(1×1, reduce) → BN → ReLU → Conv(3×3) → BN → ReLU →
Conv(1×1, expand) → BN → (+skip) → ReLU

- Step 1: 1×1 conv reduces channels: $256 \rightarrow 64$ (bottleneck)
- Step 2: 3×3 conv processes at reduced channels: $64 \rightarrow 64$
- Step 3: 1×1 conv restores channels: $64 \rightarrow 256$
- Parameters: $1 \times 1 \times 256 \times 64 + 3 \times 3 \times 64 \times 64 + 1 \times 1 \times 64 \times 256 = 69,632$ (vs 589,824 for straight 3×3)

Why bottleneck?

- ~8.5× fewer parameters than two 3×3 convs at same channel width
- Enables MUCH deeper networks (50, 101, 152 layers) without exploding parameter count
- The 1×1 convolutions add extra non-linearities (ReLU after each)

KEY TAKEAWAY: ResNet-18/34 uses Basic Blocks (3×3→3×3); ResNet-50+ uses Bottleneck Blocks (1×1→3×3→1×1) — much more efficient.

Q14. [Medium] [3 marks]

Why did VGGNet use only 3×3 convolutions? Show numerically that two 3×3 convs have the same receptive field as one 5×5 but fewer parameters.

✓ DETAILED SOLUTION

Receptive Field Equivalence:

- A single 5×5 kernel: Each output neuron looks at a $5 \times 5 = 25$ -pixel region of the input
- One 3×3 kernel: Each output neuron looks at a $3 \times 3 = 9$ -pixel region
- Two stacked 3×3 kernels: The second layer's 3×3 kernel looks at a 3×3 region of the first output, which corresponds to a 5×5 region of the original input!

Receptive field after n stacked 3×3 layers = $2n+1 \times 2n+1$

$n=1$: 3×3 | $n=2$: 5×5 | $n=3$: 7×7 | $n=4$: 9×9

Parameter Count Comparison (with C channels):

Single 5×5 conv: $5 \times 5 \times C \times C = 25C^2$

Two 3×3 convs: $2 \times (3 \times 3 \times C \times C) = 18C^2$

Savings: $25C^2 - 18C^2 = 7C^2 \rightarrow 28\%$ fewer parameters!

Three 3×3 vs One 7×7:

Single 7×7: $49C^2$ vs Three 3×3: $27C^2 \rightarrow 45\%$ saving!

Additional Benefit — More Non-linearity:

- 5×5: 1 ReLU activation per 25 input values
- Two 3×3: 2 ReLU activations — more expressive function learned
- This is why small kernels (3×3) are the industry standard today

KEY TAKEAWAY: Two 3×3 convs = same receptive field as 5×5, but use $18C^2$ params vs $25C^2$ — a 28% saving with MORE non-linearity.

Q15. [Hard] [4 marks]

Explain Inception modules and the role of 1×1 convolutions. Why is GoogLeNet dramatically smaller than VGG despite being equally accurate?

✓ DETAILED SOLUTION

The Core Problem Inception Solves:

- What kernel size should we use? 1×1? 3×3? 5×5? The answer: USE ALL OF THEM in parallel!

Basic Inception Module:

Input $\rightarrow$ [1×1 conv] + [3×3 conv] + [5×5 conv] + [MaxPool 3×3] $\rightarrow$ Concat $\rightarrow$ Output

- Each branch processes the same input at different scales
- Concatenation fuses all scales: output has all_channels = sum of branches

Problem: Computational Explosion

Direct 5×5 conv on 28×28×256 input to 32 output channels: $5 \times 5 \times 256 \times 32 \times 28 \times 28 \approx 1.2\text{B}$ multiply-adds per forward pass.

Solution: 1×1 Bottleneck Before Expensive Operations

Input (256ch) $\rightarrow$ 1×1 conv (16ch) $\rightarrow$ 5×5 conv (32ch) $\rightarrow$ Output

- Step 1 (1×1): $256 \rightarrow 16$ channels: $1 \times 1 \times 256 \times 16 \times 28 \times 28 \approx 3.2\text{M}$ ops
- Step 2 (5×5): $16 \rightarrow 32$ channels: $5 \times 5 \times 16 \times 32 \times 28 \times 28 \approx 10\text{M}$ ops
- Total: $\sim 13.2\text{M}$ ops vs 1.2B — 91× REDUCTION in computation!

Why GoogLeNet (~5M params) << VGG (~138M params):

1. 1×1 bottlenecks drastically reduce channels before expensive convolutions
 2. Global Average Pooling replaces 3 FC layers (AlexNet FC alone was $\sim 37\text{M}$ params)
 3. Efficient multi-branch design vs VGG's uniform sequential architecture
- Despite 27× fewer params, GoogLeNet achieves 6.7% vs VGG's 7.3% ImageNet error

KEY TAKEAWAY: 1×1 bottleneck reduces channels before expensive operations. Multi-scale processing. GAP replaces large FC layers.

TOPIC 6: Normalization & Regularization

NORMALIZATION & REGULARIZATION

BatchNorm · LayerNorm · Dropout · L1/L2 Regularization

Q16. [Easy] [2 marks]

What does Batch Normalization do? Write the formula for the normalized output.

✓ DETAILED SOLUTION

The Problem BN Solves:

- As weights update during training, the distribution of inputs to each layer keeps changing — this is called 'Internal Covariate Shift'
- Layers constantly need to adapt to new input distributions → slow training

BatchNorm Formula:

$$\begin{aligned}\mu_B &= (1/m) \sum_i x_i && \text{[batch mean]} \\ \sigma_B^2 &= (1/m) \sum_i (x_i - \mu_B)^2 && \text{[batch variance]} \\ \hat{x}_i &= (x_i - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)} && \text{[normalize]} \\ y_i &= \gamma \cdot \hat{x}_i + \beta && \text{[scale \& shift]}\end{aligned}$$

Variable Meanings:

- m : mini-batch size; x_i : activation of neuron i
- ϵ : small constant ($\sim 1e-8$) to prevent division by zero
- γ (gamma): learnable scale — initialized to 1
- β (beta): learnable shift — initialized to 0

Effect: Activations are standardized → training is more stable and faster

KEY TAKEAWAY: BatchNorm normalizes activations to zero mean, unit variance per feature across the batch — then rescales with γ, β .

Q17. [Medium] [3 marks]

Why does Dropout act as a regularizer? Explain the 'ensemble interpretation' of Dropout.

✓ DETAILED SOLUTION

What Dropout Does:

- During training: Each neuron is independently set to 0 with probability p (typically 0.5)
- During inference: All neurons are active; their outputs are scaled by $(1-p)$ [standard] or the training outputs are scaled by $1/(1-p)$ [inverted dropout, used in PyTorch]

Why It Regularizes — Mechanism 1: Prevents Co-adaptation

- Without dropout: Neuron A may only activate in combination with Neurons B and C (co-adaptation)
- With dropout: B or C may be dropped at any time, so A must learn to be useful independently
- Result: Each neuron learns more robust, generally useful features

Why It Regularizes — Mechanism 2: Ensemble Interpretation

- With N neurons, each can be included or excluded → 2^N possible sub-networks

- Every training batch trains a different sub-network (sampled from 2^N possibilities)
- All sub-networks share the same weights (weight tying)
- At inference, using all neurons approximates averaging predictions of all 2^N sub-networks
- Ensembles consistently outperform individual models by averaging out variance

Practical Note:

- $p=0.5$ for FC layers (standard from AlexNet)
- $p=0.2-0.3$ for conv layers (less common; conv layers already have weight sharing)
- ALWAYS call `model.eval()` at test time — dropout is disabled automatically

KEY TAKEAWAY: Dropout trains 2^N sub-networks via weight sharing. Inference approximates averaging all sub-networks.

Q18. [Hard] [4 marks]

Compare BatchNorm and LayerNorm. Provide a concrete example showing why LayerNorm is preferred for Transformers.

✓ DETAILED SOLUTION**Core Difference:**

- BatchNorm: normalizes across the BATCH dimension (per feature, across all samples)
- LayerNorm: normalizes across the FEATURE dimension (per sample, across all features)

BatchNorm — Training vs Inference Discrepancy:

- Training: μ, σ^2 computed from current mini-batch
- Inference: μ, σ^2 from running averages collected during training
- Problem: Running averages may not match test distribution — performance drop possible

BatchNorm — Small Batch Problem:

Batch size = 2 samples, 4 features: $\mathbf{x} = [[1,2,3,4], [5,6,7,8]]$
 μ_B (per feature) = [3.0, 4.0, 5.0, 6.0] (mean of 2 samples per feature)

- With only 2 samples, μ and σ^2 are noisy — poor estimates of true statistics
- With `batch_size=1` (e.g., some inference scenarios): BN is undefined ($\sigma^2=0$ for single sample)

LayerNorm — Consistent Computation:

Same sample $\mathbf{x} = [1,2,3,4]$: $\mu_L = 2.5, \sigma^2_L = 1.25$
 $\hat{\mathbf{x}} = [(1-2.5)/\sqrt{1.25}, (2-2.5)/\sqrt{1.25}, (3-2.5)/\sqrt{1.25}, (4-2.5)/\sqrt{1.25}]$
 $= [-1.34, -0.45, +0.45, +1.34]$

- Same computation at training AND inference — no discrepancy
- Works with `batch_size=1`

Why Transformers Need LayerNorm:

- Sequences have VARIABLE LENGTH: batch of sentences has different token counts
- BN would need to handle different sequence positions differently — impractical
- LN normalizes each token's representation independently — works perfectly
- All major Transformers (BERT, GPT, ViT) use LayerNorm as the default

KEY TAKEAWAY: BN depends on batch statistics — fails for small batches and variable-length sequences. LN is instance-independent.

TOPIC 7: Recurrent Neural Networks & LSTM

RNNS & LSTM

Vanilla RNN · LSTM Gates · Cell State · GRU · BPTT

Q19. [Easy] [2 marks]

What are the 4 components (gates/states) of an LSTM? What does each one do?

✓ DETAILED SOLUTION

1. Forget Gate (f_t) — 'What to ERASE from memory'

$$f_t = \sigma(W_f \cdot h_{t-1} + U_f \cdot x_t + b_f)$$

- Output range: (0, 1). $f_t \approx 0 \rightarrow$ forget; $f_t \approx 1 \rightarrow$ keep
- Example: When reading a new sentence, forget the gender of the previous subject

2. Input Gate (i_t) + Candidate Values ($\tilde{c}_t$) — 'What to WRITE to memory'

$$i_t = \sigma(W_i \cdot h_{t-1} + U_i \cdot x_t + b_i)$$

$$\tilde{c}_t = \tanh(W_c \cdot h_{t-1} + U_c \cdot x_t + b_c)$$

- i_t decides HOW MUCH of the new information to store (gate, range 0-1)
- $\tilde{c}_t$ provides the CONTENT of what to store (range -1 to 1, using tanh)

3. Cell State (c_t) — 'The Long-Term Memory Highway'

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

- Additive update (not multiplicative!) $\rightarrow$ gradients can flow unimpeded
- Forget old info + add new info = updated long-term memory

4. Output Gate (o_t) — 'What to READ from memory'

$$o_t = \sigma(W_o \cdot h_{t-1} + U_o \cdot x_t + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

- Filters which parts of the cell state to expose as the output h_t
- h_t is both the hidden state (passed to next step) and the output (for prediction)

KEY TAKEAWAY: LSTM has: Forget Gate (erase), Input Gate (write), Cell State (memory), Output Gate (read), Hidden State (output).

Q20. [Medium] [3 marks]

Trace through one LSTM time step with specific values. Input: $x_t=1.0$, $h_{t-1}=0.0$, $c_{t-1}=0.0$. Simplify: all $W, U=0.5$, all biases=0.

✓ DETAILED SOLUTION

Given: $x_t=1.0$, $h_{t-1}=0.0$, $c_{t-1}=0.0$, all $W=U=0.5$, $b=0$

Step 1 — Forget Gate

$$f_t = \sigma(0.5 \times 0.0 + 0.5 \times 1.0 + 0) = \sigma(0.5) = 1 / (1 + e^{-0.5}) \approx 0.622$$

- $\rightarrow$ 62.2% of previous cell state is retained (but $c_{t-1}=0$, so no effect here)

Step 2 — Input Gate

$$i_t = \sigma(0.5 \times 0.0 + 0.5 \times 1.0 + 0) = \sigma(0.5) \approx 0.622$$

- 62.2% of candidate information will be written to memory

Step 3 — Candidate Values

$$\tilde{c}_t = \tanh(0.5 \times 0.0 + 0.5 \times 1.0 + 0) = \tanh(0.5) \approx 0.462$$

Step 4 — Cell State Update

$$\begin{aligned} c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ &= 0.622 \times 0.0 + 0.622 \times 0.462 \\ &= 0.0 + 0.287 = 0.287 \end{aligned}$$

- Memory went from 0 to 0.287 — the LSTM learned something from $x_t=1.0$!

Step 5 — Output Gate

$$o_t = \sigma(0.5 \times 0.0 + 0.5 \times 1.0) = \sigma(0.5) \approx 0.622$$

Step 6 — Hidden State

$$h_t = o_t \odot \tanh(c_t) = 0.622 \times \tanh(0.287) = 0.622 \times 0.279 \approx 0.174$$

RESULT: Starting from scratch ($h_0 = c_0 = 0$), after one step with input 1.0: cell memory = 0.287, hidden state output = 0.174.

KEY TAKEAWAY: Concrete LSTM trace shows how gates control information flow. Both forget and input gates start at ~0.5.

Q21. [Hard] [4 marks]

A vanilla RNN is trained on sequences of length 100. Explain quantitatively why it fails to learn dependencies between token 1 and token 100.

✓ DETAILED SOLUTION

Vanilla RNN Gradient Flow (BPTT):

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$$

Gradient from loss L at step T to hidden state at step 1:

$$\partial L / \partial h_1 = \partial L / \partial h_T \times (\partial h_T / \partial h_{T-1}) \times \dots \times (\partial h_2 / \partial h_1)$$

$$\text{Each factor: } \partial h_t / \partial h_{t-1} = \text{diag}[\tanh'(z_t)] \times W_{hh}$$

Quantitative Analysis:

- $\tanh'(z) = 1 - \tanh^2(z)$, which is ≤ 1 always, and ≤ 0.5 for $|z| > 0.58$
- Eigenvalues of W_{hh} control whether gradients vanish or explode
- If $\lambda_{\max}(W_{hh}) < 1$: gradient shrinks exponentially
- For a typical RNN: $\|\partial h_t / \partial h_{t-1}\| \approx 0.5$ (generous estimate)

$$\text{Gradient magnitude after 99 steps: } 0.5^{99} \approx 1.6 \times 10^{-30}$$

$$\text{With sigmoid derivative } \leq 0.25: \quad 0.25^{99} \approx 6 \times 10^{-60}$$

Practical Consequence:

- The gradient of the loss w.r.t. W at step 1 is NUMERICALLY ZERO in floating point
- w_1 receives no update — it never learns to encode information relevant to the final output
- The RNN effectively ignores inputs more than ~10-20 steps ago

LSTM Solution:

- Cell state update: $c_t = f_t \odot c_{t-1} + \dots$ (additive, not multiplicative)

$$\partial c_t / \partial c_{t-1} = f_t \quad (\text{diagonal, values} \approx 1 \text{ when forget gate is open})$$

- The 'gradient highway' allows error signals to flow back 100+ steps without vanishing

KEY TAKEAWAY: Gradient at step 1 = product of 99 Jacobians. With sigmoid, this is at best $0.25^{99} \approx 10^{-60}$ — mathematically zero.

TOPIC 8: Sequence Models & Advanced RNNs

SEQUENCE MODELS

GRU · Bidirectional RNN · Seq2Seq · Attention · Teacher Forcing

Q22. [Easy] [2 marks]

What is the difference between many-to-one and many-to-many RNN architectures? Give a real-world example of each.

✓ DETAILED SOLUTION

Many-to-One (Sequence → Single Output)

- Input: sequence of tokens $x_1, x_2, \dots, x_T$
- Output: single label/value y (using only the last hidden state h_T)
- Example 1: Sentiment Analysis — 'The movie was absolutely fantastic!' → POSITIVE
- Example 2: Spam detection — email text → spam/not spam
- Example 3: Stock price prediction — last 30 days of prices → tomorrow's price

Many-to-Many (Sequence → Sequence)

- Two sub-types:
- a) SYNCHRONIZED: Output at each input step. Example: POS tagging (noun/verb per word)
- b) ENCODER-DECODER: Read full input, then generate output. Example: Machine Translation
- English → French: 'The cat sat on the mat' → 'Le chat était assis sur le tapis'
- Source and target can have different lengths!

One-to-Many (Single Input → Sequence)

- Example: Image Captioning — one image → sequence of words describing it
- CNN encodes image → LSTM decodes to caption word by word

KEY TAKEAWAY: Many-to-one: sequence → label (sentiment). Many-to-many: sequence → sequence (translation).

Q23. [Medium] [3 marks]

What is Teacher Forcing in sequence models? What is the problem it introduces, and what is the solution?

✓ DETAILED SOLUTION

What is Teacher Forcing?

During training of a sequence-to-sequence model (e.g., image captioning, machine translation), the ground truth token from time step t is used as the INPUT to time step $t+1$, rather than the model's own prediction.

- WITHOUT teacher forcing: $\text{pred}(t) \rightarrow \text{input}(t+1)$ [autoregressive]
- WITH teacher forcing: $y_{\text{true}}(t) \rightarrow \text{input}(t+1)$

Why Use It?

- Without teacher forcing: If model makes one mistake (wrong word), error compounds — all subsequent predictions are built on the wrong foundation. Training diverges or is very slow.
- With teacher forcing: Each step has a correct foundation. Training is fast and stable.

The Problem — Exposure Bias:

- During TRAINING: model always sees correct previous tokens (teacher forcing)
- During INFERENCE: model sees its OWN predictions (which may be imperfect)
- The model is never trained on the kind of input it sees at test time!
- Result: Errors accumulate at inference time; model can produce nonsensical outputs

Solution — Scheduled Sampling (Bengio et al., 2015):

- Gradually transition from teacher forcing to self-prediction during training

$P(\text{use ground truth}) = \epsilon_i$ where ϵ_i decreases over training iterations

- Early training: $\epsilon \approx 1.0$ (always use ground truth — stable)
- Late training: $\epsilon \approx 0.0$ (always use own prediction — realistic)
- This bridges the gap between training conditions and inference conditions

KEY TAKEAWAY: Teacher forcing speeds training but causes exposure bias. Scheduled sampling gradually bridges train/test gap.

Q24. [Hard] [4 marks]

Design a complete image captioning system. Explain encoder, decoder, training, and inference with all technical details.

✓ DETAILED SOLUTION**ENCODER: CNN Feature Extractor**

- Use pre-trained ResNet-50/101 (ImageNet pre-trained)
- Remove final classification layer
- Apply Global Average Pooling → 2048-dim feature vector

```
v_img = GAP(ResNet(image)) → shape: (batch, 2048)
```

- Project to embedding space: $v_embed = \text{Linear}(2048, \text{hidden_dim})$
- Use v_embed as the initial hidden state h_0 of the LSTM decoder

DECODER: LSTM Word Generator

- Vocabulary: {<pad>, <start>, <end>, all unique words}
- Input embedding: $\text{Embedding}(\text{vocab_size}, \text{embed_dim})$

```
At step t: h_t = LSTM(embed(word_{t-1}), h_{t-1})
logits_t = Linear(h_t, vocab_size)
word_t = argmax(softmax(logits_t))
```

TRAINING: Teacher Forcing + Cross-Entropy

- Feed <start> token → predict word 1 (using ground truth caption)
- Feed ground truth word 1 → predict word 2 (teacher forcing)
- Continue until <end> token or max_length

```
Loss = CrossEntropy(predicted_tokens, ground_truth_tokens)
```

INFERENCE: Greedy Decoding / Beam Search

- Greedy: At each step, pick $\text{argmax}(\text{softmax}(\text{logits}))$ — fast but suboptimal
- Beam Search ($k=5$): Keep top-5 most likely sequences at each step
- Stop: when <end> token generated or max_length reached

EVALUATION METRIC:

- BLEU score: measures n-gram overlap between predicted and reference captions

- BLEU-4 (4-gram) is the standard metric for captioning benchmarks (MS-COCO)

KEY TAKEAWAY: *CNN encoder $\rightarrow$ feature vector $\rightarrow$ LSTM decoder. Teacher forcing in training; beam search in inference.*

TOPIC 9: Autoencoders & Image Segmentation

AUTOENCODERS & IMAGE SEGMENTATION

AE Variants · U-Net · SegNet · DeepLab · Loss Functions · Metrics

Q25. [Easy] [2 marks]

What is an autoencoder? What loss function is used for grayscale image reconstruction vs binary data?

✓ DETAILED SOLUTION

Autoencoder Architecture:

Input $X \rightarrow$ Encoder $f \rightarrow$ Latent $z \rightarrow$ Decoder $g \rightarrow$ Reconstruction $\hat{X}$

- Encoder: Compresses input to lower-dimensional latent representation z
- Bottleneck: z has much smaller dimension than X — forces efficient encoding
- Decoder: Reconstructs X from z — trained to minimize reconstruction error
- NO LABELS needed! Purely unsupervised learning.

Loss Functions:

- GRAYSCALE IMAGES (continuous pixel values 0-255 or 0-1):

$$L_{MSE} = (1/N) \sum_i (x_i - \hat{x}_i)^2$$

- Binary/bit data (0 or 1 values):

$$L_{BCE} = -\sum_i [x_i \log(\hat{x}_i) + (1-x_i) \log(1-\hat{x}_i)]$$

- For binary segmentation masks $\rightarrow$ also use BCE

Applications:

- Dimensionality reduction (alternative to PCA, but non-linear)
- Feature learning for downstream tasks
- Anomaly detection: high reconstruction error = anomaly
- Data compression, denoising (denoising AE variant)

KEY TAKEAWAY: AE = Encoder (compress) + Decoder (reconstruct). MSE for continuous; BCE for binary inputs.

Q26. [Medium] [3 marks]

Compare U-Net and SegNet architectures for image segmentation. What is the key structural difference in their decoders?

✓ DETAILED SOLUTION

U-Net Architecture:

- Encoder: 4 downsampling blocks (Conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool). Features: 64 $\rightarrow$ 128 $\rightarrow$ 256 $\rightarrow$ 512.
- Bottleneck: 1024 channels at smallest spatial resolution
- Decoder: 4 upsampling blocks (UpSample $\rightarrow$ Concatenate_skip $\rightarrow$ Conv $\rightarrow$ BN $\rightarrow$ ReLU)
- Skip connections: COPY entire feature maps from encoder to decoder (NOT just indices)

Decoder input = Upsample($h_{decoder}$) CONCAT Encoder_feature_map

SegNet Architecture:

- Encoder: Same as U-Net (Conv+MaxPool)
- KEY DIFFERENCE: During MaxPool, SAVE the indices of max values (not the feature maps)
- Decoder: Use stored indices to place values back in correct positions (max-unpooling)

MaxUnpool: places activation at original max position, zeros elsewhere

Comparison Table:

- Memory usage: U-Net > SegNet (U-Net stores full feature maps; SegNet only stores indices)
- Boundary accuracy: U-Net > SegNet (full feature maps restore more spatial detail)
- Speed: SegNet faster (lighter decoder, no concatenation overhead)
- Use U-Net for: medical imaging, tasks requiring precise boundaries
- Use SegNet for: real-time applications, embedded/edge devices

KEY TAKEAWAY: U-Net copies full feature maps via skip connections; SegNet only stores pooling indices — memory vs accuracy tradeoff.

Q27. [Hard] [4 marks]

Explain DeepLab's Atrous Spatial Pyramid Pooling (ASPP). How does it handle objects at different scales?

✓ DETAILED SOLUTION**The Multi-Scale Problem:**

- A large dog: needs a large receptive field to understand the whole object
- A small bird: needs fine-grained local detail and small receptive field
- Single-scale CNN fails at one of these — ASPP solves this by using BOTH simultaneously

ASPP Module Structure:

```

Input feature map (from backbone CNN)

Branch 1: 1×1 conv (rate=1) → effective field: 1×1 [local context]
Branch 2: 3×3 conv (rate=6) → effective field: (3-1)×6+1 = 13×13
Branch 3: 3×3 conv (rate=12) → effective field: (3-1)×12+1 = 25×25
Branch 4: 3×3 conv (rate=18) → effective field: (3-1)×18+1 = 37×37
Branch 5: Global Avg Pool → upsample → [global scene context]
Concatenate all branches → 1×1 conv to fuse → segmentation output

```

Why This Works:

- Each branch uses the SAME feature map but captures DIFFERENT scales of context
- Branch 1 detects fine local details (textures, small objects)
- Branches 2-4 detect medium and large objects in context
- Branch 5 captures global scene information (what kind of scene is this?)
- All use DILATION — no stride → full spatial resolution maintained!

CRF Post-Processing (DeepLabv2):

$$E(\mathbf{x}) = \sum_i \theta_i(\mathbf{x}_i) + \sum_{i,j} \theta_{ij}(\mathbf{x}_i, \mathbf{x}_j)$$

- Unary: CNN prediction per pixel (what class is this pixel?)
- Pairwise: Similar color pixels nearby should have same label
- Mean-field optimization refines boundaries iteratively (3-10 iterations)
- Result: Sharper boundaries, less noise in predictions

KEY TAKEAWAY: ASPP applies dilated convolutions at multiple rates (1, 6, 12, 18) in parallel, capturing features from 1×1 to 37×37 effective regions.

TOPIC 10: PyTorch Code & Tracing Questions

PYTORCH CODE & TRACING

Gradient Computation · Dimension Tracing · Training Errors · Parameter Count

Q28. [Easy] [2 marks]

What is the output of the following code? Explain why.

✓ DETAILED SOLUTION

Code:

```
import torch
x = torch.tensor(2.0, requires_grad=True)
y = x ** 3
y.backward()
print(x.grad) # Output: ?
```

Solution:

$$y = x^3 \rightarrow dy/dx = 3x^2 \rightarrow \text{At } x=2: 3 \times 4 = 12$$

Output: tensor(12.)

Extension — What if we call backward() twice?

```
y2 = x ** 2
y2.backward()
print(x.grad) # Output: 12 + 4 = 16 (ACCUMULATED!)
```

- PyTorch ACCUMULATES gradients. $x.grad = 12$ (from y) + 4 (from $y2$) = 16
- This is why `optimizer.zero_grad()` or `x.grad.zero_()` must be called before each training step
- Exception where accumulation is useful: gradient checkpointing, large models with microbatches

KEY TAKEAWAY: PyTorch grad accumulates by default. `zero_grad()` must clear before each backward pass.

Q29. [Medium] [3 marks]

What does the following training code output? Predict pred, loss, gradients, and new weights after one step.

✓ DETAILED SOLUTION

Code:

```
model = nn.Linear(1, 1, bias=True)
model.weight.data.fill_(1.0) # w = 1.0
model.bias.data.fill_(0.5) # b = 0.5
optimizer = SGD(model.parameters(), lr=0.5)
x = torch.tensor([[2.0]]) ; target = torch.tensor([[3.0]])
pred = model(x) ; loss = (pred - target)**2
optimizer.zero_grad() ; loss.backward() ; optimizer.step()
```

Step 1 — Forward Pass:

$$\text{pred} = w \times x + b = 1.0 \times 2.0 + 0.5 = 2.5$$

Step 2 — Loss:

$$\text{loss} = (\text{pred} - \text{target})^2 = (2.5 - 3.0)^2 = (-0.5)^2 = 0.25$$

Step 3 — Gradients:

$$\partial \text{loss} / \partial \text{pred} = 2(\text{pred} - \text{target}) = 2(-0.5) = -1.0$$

$$\partial \text{loss} / \partial w = \partial \text{loss} / \partial \text{pred} \times \partial \text{pred} / \partial w = -1.0 \times x = -1.0 \times 2.0 = -2.0$$

$$\partial \text{loss} / \partial b = \partial \text{loss} / \partial \text{pred} \times \partial \text{pred} / \partial b = -1.0 \times 1.0 = -1.0$$

Step 4 — SGD Update (lr=0.5):

$$w_{\text{new}} = w - \text{lr} \times \text{grad}_w = 1.0 - 0.5 \times (-2.0) = 1.0 + 1.0 = 2.0$$

$$b_{\text{new}} = b - \text{lr} \times \text{grad}_b = 0.5 - 0.5 \times (-1.0) = 0.5 + 0.5 = 1.0$$

Final Output:

```
pred: 2.5 | loss: 0.25 | w.grad: -2.0 | b.grad: -1.0
```

```
new_w: 2.0 | new_b: 1.0
```

KEY TAKEAWAY: Trace: pred=2.5, loss=0.25, w.grad=-2.0, b.grad=-1.0, new_w=2.0, new_b=1.0

Q30. [Hard] [4 marks]

The following CNN crashes for input (4,3,64,64). Find the bug, explain why it crashes, and propose two different fixes.

✓ DETAILED SOLUTION**Buggy Code:**

```
class BuggyNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 32, k=3, s=2, p=1), # Layer 1
            nn.ReLU(),
            nn.MaxPool2d(2, 2), # Layer 2
        )
        self.fc = nn.Linear(32*6*6, 10) # ASSUMES 6x6
    def forward(self, x):
        x = self.conv(x)
        x = x.flatten(1)
        return self.fc(x) # CRASHES HERE
```

Tracing dimensions for input (4, 3, 64, 64):

After Conv2d(k=3,s=2,p=1): $H = \text{floor}((64+2-3)/2)+1 = \text{floor}(63/2)+1 = 32$

Shape: (4, 32, 32, 32)

After MaxPool2d(k=2,s=2): $H = \text{floor}((32-2)/2)+1 = 16$

Shape: (4, 32, 16, 16)

Flatten: $32 \times 16 \times 16 = 8192$ features

fc expects: $32 \times 6 \times 6 = 1152$ features ← MISMATCH → RuntimeError!

FIX 1: Adaptive Average Pooling (RECOMMENDED):

```
self.gap = nn.AdaptiveAvgPool2d((6, 6)) # Forces output to 6x6
# In forward: x = self.gap(self.conv(x))
```

- Benefit: Works with ANY input size — production-safe!

- The network automatically adapts to different resolutions

FIX 2: Recompute fc input size:

```
self.fc = nn.Linear(32*16*16, 10) # Use correct 16×16
```

- Simpler fix, but ONLY works for exactly 64×64 inputs
- Best practice: Always calculate exact dimensions before writing fc layer

KEY TAKEAWAY: *The Linear layer expects $32 \times 6 \times 6 = 1152$ inputs but gets $32 \times 7 \times 7 = 1568$. Fix with AdaptivePool or recompute dims.*

EXAM QUICK REFERENCE**EXAM QUICK REFERENCE***Key Formulas & Facts to Memorize*

Topic	Formula / Key Fact
Output Size (Conv)	$\text{floor}((\text{In} - K + 2P) / S) + 1$
Effective Kernel (Dilation)	$(K - 1) \times \text{rate} + 1$
Params (Conv2d)	$K \times K \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$
Params (Linear)	$\text{in_features} \times \text{out_features} + \text{out_features}$
Sigmoid derivative	$\sigma(x)(1-\sigma(x)) \leq 0.25$
ReLU derivative	$1 \text{ if } x > 0, \text{ else } 0$
Softmax	$e^{y_i} / \sum_j e^{y_j}$
Cross-Entropy Loss	$-\sum y_i \log(\hat{y}_i)$
Dice Loss	$1 - (2\Sigma p + \epsilon) / (\Sigma p + \Sigma g + \epsilon)$
LSTM Cell State	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$
LSTM Hidden State	$h_t = o_t \odot \tanh(c_t)$
GRU Hidden State	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
BatchNorm	$\hat{x} = (x - \mu_B) / \sqrt{(\sigma^2 + \epsilon)}, \quad y = \gamma \hat{x} + \beta$
Adam Update	$w = w - \alpha \cdot m^{\wedge} / (\sqrt{v^{\wedge}} + \epsilon)$
IoU	$TP / (TP + FP + FN)$
Dice $\leftrightarrow$ IoU	$\text{Dice} = 2 \times \text{IoU} / (1 + \text{IoU})$
ResNet Output	$F(x) + x \quad (\text{residual} + \text{skip})$

□ EXAM STRATEGY:

1. Always write the formula FIRST, then substitute values
2. Show UNITS and SHAPE at each step in dimension tracing questions
3. For code questions: trace forward pass manually before computing gradients
4. For 'explain' questions: Intuition $\rightarrow$ Formula $\rightarrow$ Example $\rightarrow$ Edge Case
5. Common traps: dying ReLU, eval() mode, zero_grad(), BN train vs inference